

UNIT-II

CPU SCHEDULING

CPU scheduling :-

Scheduling is not but it is a way of selecting the process from Ready queue & put the process in CPU, is called CPU scheduling.

CPU scheduling is two types

- 1) Preemptive Preemptive
- 2) Non-preemptive Non-preemption

Preemptive :-

When there is an interrupt occurs then it is called preemptive

Non-preemptive :-

When there is no interrupt occur then it is called Non-preemptive

CPU Scheduling criteria :-

- 1) CPU utilization :-

It means how long CPU is busy.

- 2) throughput :-

That means no. of instructions are executed in particular time.

- 3) Arrival time (AT) :-

It tells at what time the process is entered in CPU

4) Burst time (BT)

It tells how much time taken by CPU to execute the process.

5) Turn-Around time (TAT)

How much time a process take for execution.

6) Waiting time

How much time the process wait for the allocation of CPU.

Note:-

$$(AT + BT)$$

$$TAT = \text{Completion time} - \text{Arrival time}$$

$$(CT) \quad (AT)$$

CT :- At what time it is allocating or executing is known as completion time.

7) FCFS (First come First Serve)

It is a non-preemptive algorithm, it is based on arrival time only.

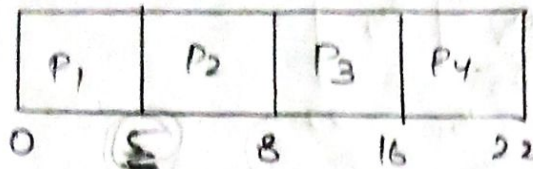
	(Seconds) AT	(Seconds) BT	(Seconds) CT	(Seconds) TAT (CT-AT)	(TAT-BT) Wait time
P ₁	0	5	5	5	0
P ₂	1	3	8	7	4
P ₃	2	8	16	14	6
P ₄	3	6	22	19	13

$$\text{Total} = 0 + 4 + 6 + 13 = 23$$

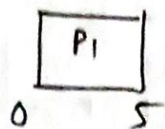
$$\text{Total} = \frac{23}{4} = 5.75$$

No. of process = 4

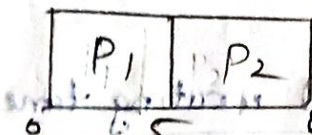
Gantt chart



Step-1:- At 0 sec, the P₁ entered & it takes 5 sec to execute

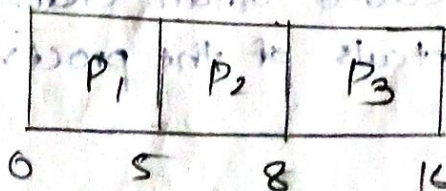


Step-2:- At this situation, we have 3 processes in the queue. Based on Arrival time, P₂ is going to be allocated to CPU.



Step-3:- Now P₂ will take three seconds, to execute i.e. $5 + 3 = 8$

Step-4:- Now P₃ will enter in ready queue and allocated to CPU, so, it enter at 8 sec & it takes 8 sec to execute i.e. $8 + 8 = 16$



Step-5 :- Now P_4 will enter to ready queue and allocate to CPU. So it enters at 16 sec & take 6 seconds to execute. $16 + 6 = 22 \text{ sec}$

P_1	P_2	P_3	P_4	
0	5	8	16	22

To calculate the CT we have to consider the entering time and executing time.

5 8 16 22

TAT :- CT - AT

WT :- TAT - BT

To calculate the total average time ^{waiting}

$$\text{Total} :- 0 + 4 + 6 + 13 = 23$$

Average waiting time = $\frac{\text{Total waiting time}}{\text{No. of process}}$

$$= \frac{23}{4} = 5.75$$

$$\text{Total TAT} = 5 + 7 + 14 + 19 = 45$$

$$\text{Avg} = \frac{\text{TAT}}{4} = \frac{45}{4} = 11.2$$

The process table is called Gantt chart it consists of complete details of the process.

P_1	P_2
0	11
1	16

Consider four process P_1, P_2, P_3, P_4

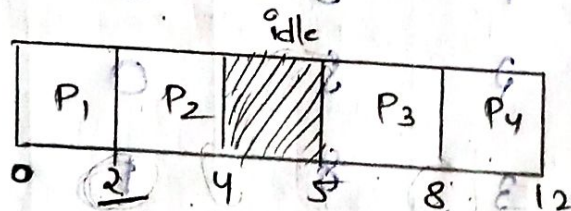
	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)
P_1	0	2	2	2	0
P_2	1	2	4	3	1
P_3	5	3	8	3	0
P_4	6	4	12	6	2

Total TAT :- $2+3+3+6=14$

Avg TAT $\neq 14/4 \Rightarrow 3.5$

Total WT :- $0+1+0+2=3$

Avg WT $\neq 3/4 \Rightarrow 0.75$



	AT	BT	CT	TAT	WT
P_1	1	10	10 11	9 10	0
P_2	2	5	15 16	13 14	9
P_3	3	4	20 23	19 20	13
P_4	4	6	28 29	24 25	19

	P_1	P_2	P_3	P_4
--	-------	-------	-------	-------

0 1 10 16 23 29

$$\text{Total TAT} = 9 + 13 + 19 + 24 = 65$$

$$\text{Avg} = \frac{65}{4} = 16.25$$

$$\text{Total WT} = 0 + 9 + 13 + 19$$

$$= 41/4 = 10.25$$

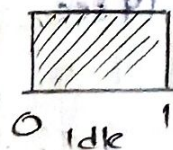
	AT	BT	(BT-AT)	(TAT-BT)
P_1	0	2	2	0
P_2	3	1	-2	0
P_3	5	2	-3	-2
P_4	7	3	-4	1

SJF (Shortest Job First)

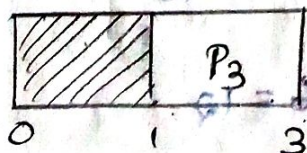
- It is based on Burst time and it is also a non-primitive algorithm. In this algorithm we have to find total turn around time (TAT) and complete time (CT) and finally waiting time (WT).
- In SJF, which process lower BT that process we need to select first.
- It can be done by using "Gantt chart". Because it makes the problem very easy.

	AT	BT	CT	TAT	WT
P ₁	1	3	6	5	2
P ₂	2	4	10	8	4
P ₃	1	2	3	2	0
P ₄	4	4	14	10	6

Step-1:- Here no process is arrived at zero, we can make CPU as idle, will be idle from 0 to 1



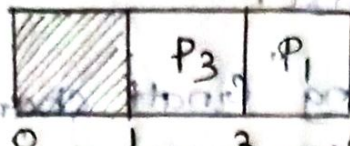
Step-2:- first we need to check at 1 sec which process has arrived, then we should check the burst time of that process.



At 1 sec, P₁ and P₃ both are arrived. Now we should check the BT of each process.

$P_2 \rightarrow 3$ } Among two process P_3 is having
 $P_3 \rightarrow 2$ } less BT.

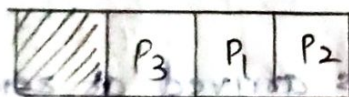
Step-3:- Now, P_3 will execute. Next, at that the
 time 3 sec, P_3 will arrive at 2 sec. So,
 P_1 will allocated to CPU.



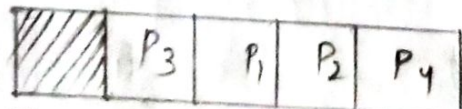
$P_2 \rightarrow 4$ } Among two process P_1 is going to
 $P_1 \rightarrow 3$ } execute because it has less burst time

Step-4:- Now P_2 will allocated to CPU at 10 sec

$P_2 \rightarrow 4$ } Among two process P_3 & P_4 having
 $P_4 \rightarrow 4$ } same BT but AT is less in
 P_2 , so P_2 will execute



Step-5:- Now P_4 will execute at 14 sec



$$\text{Total TAT} = 5 + 8 + 2 + 10 = 25$$

$$\text{Avg TAT} = 25/4 = 6.25$$

$$\text{Total WT} = 2 + 4 + 0 + 6 = 12$$

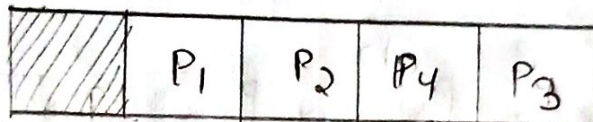
$$\text{Avg WT} = 12/4 = 3$$

Note:-

The Response time is always equal to waiting-time

	AT	BT	CT	(CT-AT) TAT	(TAT-BT) WT
P ₁	1	21	22	21	0
P ₂	2	3	25	23	20
P ₃	3	6	35	32	26
P ₄	4	4	29	25	21

Gantt chart



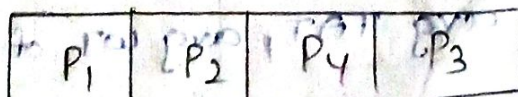
Total TAT = 101

Avg TAT = $101/4 = 25.25$

Total WT = 67

Avg WT = $67/4 = 16.75$

	AT	BT	CT	(CT-AT) TAT	(TAT-BT) WT
P ₁	0	8	8	8	0
P ₂	1	4	12	11	7
P ₃	2	9	26	24	15
P ₄	3	5	17	14	9



0 8 12 17 26

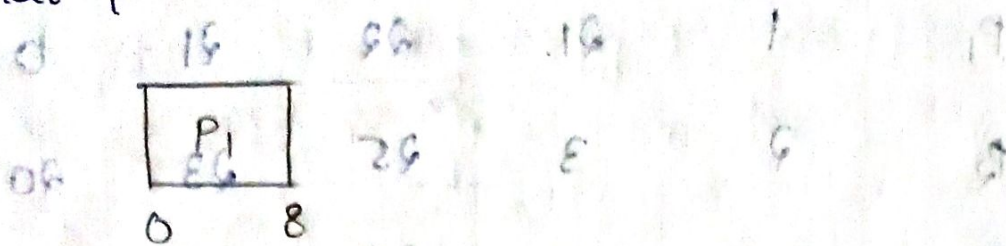
Total TAT = 57

Avg TAT = $57/4 = 14.25$

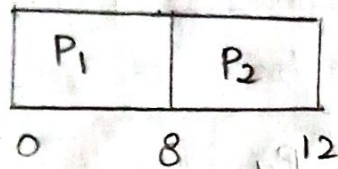
Total WT = 31

Avg WT = $31/4 = 7.75$

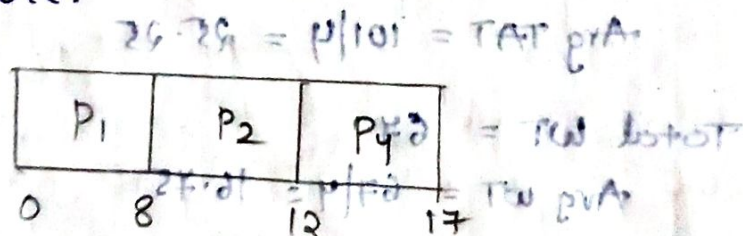
Step-1:- First we need check the AT of each process at that time 0 sec in ready queue only one process is available automatically that process will execute.



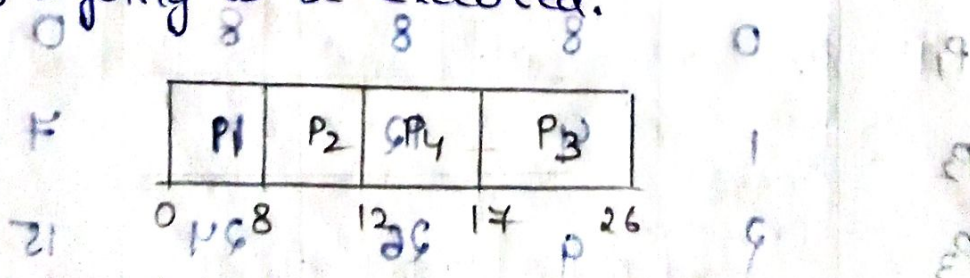
Step-2:- When the time 8 sec, all the process are in ready queue among all the process have low BT. So, P2 will execute.



Step-3:- Next, check which process have lowest BT. Among 2 process P4 have lowest BT. So P4 will execute.



Step-4:- Finally, P4 processes left in queue. Now P3 is going to be executed.



Step-5:- Calculate the CT of all processes

Step-6:- Calculate the TAT & WT of all process.

Step-7:- Find the avg TAT, avg WT of process.

SJF (Preemption):- (shortest job first)

Also called 'SRTF' (Shortest Remaining time first)
It is based on preemption (Burst-time).

Eg:- Consider 5 process

	AT	BT	CT	TAT	WT
P ₁	0	8	20	20	2
P ₂	1	2	2	1	0
P ₃	2	3	5	3	1
P ₄	3	2	7	4	2
P ₅	4	6	13	7	3

Step-1:- CPU always start from '0'. At '0' sec P₁ is going take CPU. But when time changes to 1, P₂ also available. As it is preemption whenever process come with lowest BT then that process is allocated.

Step-2:- At that time 2 sec, P₂ and P₃ are available



Step-3:- Now at 5 sec, P₄, P₅, P₁ processes are available among all process P₄ have less BT so P₄ is going to be executed.

Step-4:- Now at the time '7' second 2 process are available, P₅ is allocated to CPU

Note:- At the time 2 sec P_3 will take 1 sec to execute. It means reached to 3. So, at 3 sec both P_3 & P_4 have same BT i.e., 1 sec. Now when any two process have same BT we have to consider AT. Among 2 process (P_3, P_4) P_3 have less AT than P_4 . So, P_3 is going to be executed.

Step-5: Calculate CT, TAT, WT

$$TAT = 20 + 1 + 3 + 4 + 4 = 35$$

$$Avg = 7$$

$$WT = 12 + 0 + 11 + 2 + 3 = 18$$

$$Avg WT = 18/5 = 3.6$$

P_1	P_2	P_3	P_4	P_5	P_1
-------	-------	-------	-------	-------	-------

	AT	BT	CT	TAT	WT
P_1	0	8	21	21	13
P_2	1	1	2	1	0
P_3	2	4	8	6	2
P_4	3	2	4	1	
P_5	4	6	14	10	4

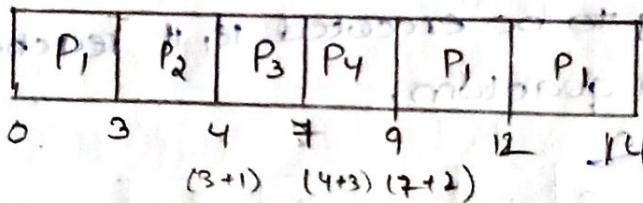
	P_1	P_2	P_4	P_3	P_5	P_1
BT	0	1	2	4	8	14
CT						

Round Robin :-

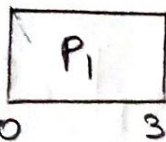
- It is designed for time-sharing systems.
- It is similar to FCFS algorithm.
- In this algorithm a small unit of time called "Time Quantum" (or) "Time slice" is defined.
- It is used for context switching between the process.

Eg:- Consider four process

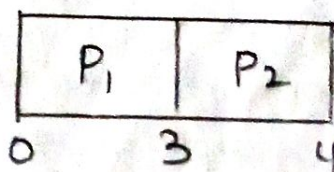
Process	AT	BT	Time Quantum	CT	CT-AT	TAT-BT
P ₁	0	8	3	14	14	6
P ₂	1	1	3	4	3	2
P ₃	2	3	3	7	5	2
P ₄	3	2	3	9	6	4



Step-1:- Initially the CPU starts from '0'. At 0 second P₁ is arrived. Now Time quantum is 3 sec only. So, P₁ will execute upto 3 seconds



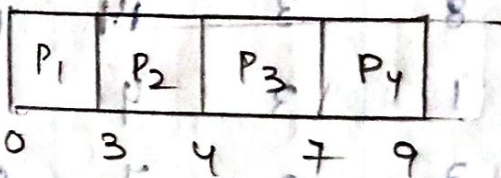
Step-2:- Now P₂ will arrive at 1 second and executed in 1 second. It means it reaches the time quantum.



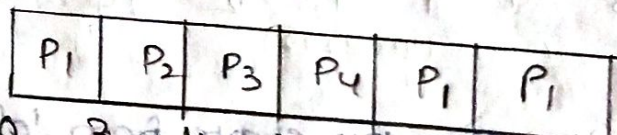
Step-3:- At 4th second, P_3 has arrived and it is executed within 3 seconds. It also reached the time quantum.



Step-4:- At 7th sec, P_4 has arrived BT of P_4 is 2 seconds ie, it is also reached the time quantum.



Step-5:- Now P_1 is left with 5 sec, of time in the queue. In the 5 sec 3 sec of time is going to be executed ie, it reached the time quantum.



Step-6:- Now P_1 will use remaining 2 sec of time for execution.

④ ③ ② ①
P₅ P₄ P₃ P₂ P₁ P₀

Process	AT	BT	Time Quantum	CT	TAT	WT
P ₀	0	5	3	18	18	13
P ₁	1	6	3	21	20	14
P ₂	2	3	3	9	7	4
P ₃	3	1	3	10	7	6
P ₄	4	5	3	23	19	14
P ₅	6	4	3	24	18	14

① ② ③ ④

P₅ P₄ P₁ P₀ P₅ P₄ P₃ P₂ P₁ P₀

P ₀	P ₁	P ₂	P ₃	P ₄	P ₅	P ₀	P ₁	P ₄	P ₅
----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------

0 3 6 9 10 13 16 18 21 23 24

$$\text{Total TAT} = 18 + 20 + 7 + 7 + 19 + 18 = 89$$

$$\text{Avg TAT} = 89 / 6 = 14.83$$

$$\text{Total WT} = 13 + 14 + 4 + 6 + 14 + 14 = 65$$

$$\text{Avg WT} = 65 / 6 = 10.83$$

Priority Scheduling (Non-preemption mode)

- It is a non-preemption algorithm in this algorithm each process is associated with priority
- It is one of the most common scheduling algorithm in batch OS.
- In this, each process is assigned to arrival time and burst time.
- Equal priority process are scheduled as FCFS (less AT process will execute first)

Process	Arrival	AT	BT	CT	TBT	ET
P ₁	0	0	9	9	9	0
P ₂	1	1	4	14	23	20
P ₃	2	2	3	7	12	4
P ₄	4	3	4	21	18	11
P ₅	5	4	3	28	24	21

Step 1: The CPU execution always starts from '0'. at '0' second the P₁ has arrived in ready queue so, that process is allocated to CPU.



Step 2: At step 2, all the processes are arrived among all processes the process which have low number is high priority. P₃ (3 priority) will allocated to CPU.



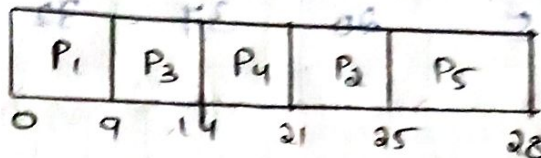
Step 3: At the time 4 sec, 3 processes are available in the ready queue, but among those processes P₂ is having '2' priority (high)



Step-4:- At 21st sec, 4 processes are available. Among those processes P_2 having high priority - 3



Step-5:- At that time of 25 sec, P_5 will be given to CPU with priority - 4.



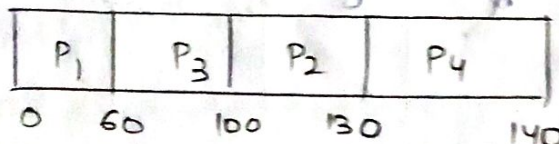
$$TAT = 9 + 24 + 12 + 18 + 24 = 87$$

$$Avg = 87/5 = 17.4$$

$$WT = 0 + 20 + 7 + 11 + 21 = 59$$

$$Avg = 59/5 = 11.8$$

Priority	Process	AT	BT	CT	TAT	WT
3	P_1	0	60	60	60	0
2	P_2	3	30	130	127	97
1	P_3	4	40	100	96	56
4	P_4	9	10	140	131	121



$$TAT = 60 + 127 + 96 + 131 = 414$$

$$Avg = 414/4 = 103.5$$

$$WT = 0 + 97 + 56 + 121 = 274$$

$$Avg = 274/4 = 68.5$$

6) Priority Scheduling :- (preemption mode)

Priority	Process	AT	BT	CT	TAT	WT
2	P ₁	0	10	17	17	7
1	P ₂	2	5	9	7	2
0	P ₃	3	2	5	2	0
3	P ₄	5	20	37	32	12

1st AT to AT then add ^{acc} to priority

P ₁	P ₂	P ₃	P ₄
0	2	3	5
	(2+1)		

$$TAT = 17 + 7 + 2 + 32 = 58$$

$$Avg = 58/4 = 14.5$$

$$WT = 7 + 2 + 0 + 12 = 21$$

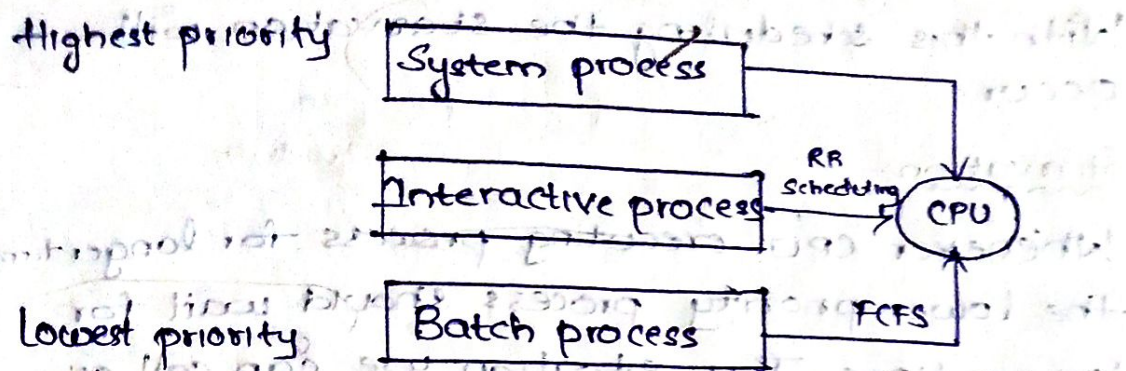
$$Avg = 21/4 = 5.25$$

Priority	Process	AT	BT	CT	TAT	WT
2	P ₁	0	9	9	9	1
1	P ₂	1	4	5	5	1
3	P ₃	2	8	10	10	1

$$TAT = 9 + 5 + 10 = 24$$

$$Avg = 24/3 = 8$$

Multilevel Queue Scheduling



- In this level of scheduling we have multiple ready^{only} queues whereas in previous algorithms we have one ready queue.
- Here, the process are stored in ready queue based on its property.
- A process property can be size, CPU time, priority etc.

→ Multilevel queue scheduling is divided into 3 queues

1) System process queue:-

In order to use OS we use some processes that processes stored in system process queue.

2) Interactive process ready queue:-

It means the response time is very fast manner.

Eg:- Any gaming application

3) Batch process:-

It means the process response time is very less. It does not consider the response time.

→ Out of all system process queue has high priority

Note:-

Here each and every queue has its own CPU scheduling algorithms Eg:- FCFS, RR, SJFS etc. priority based

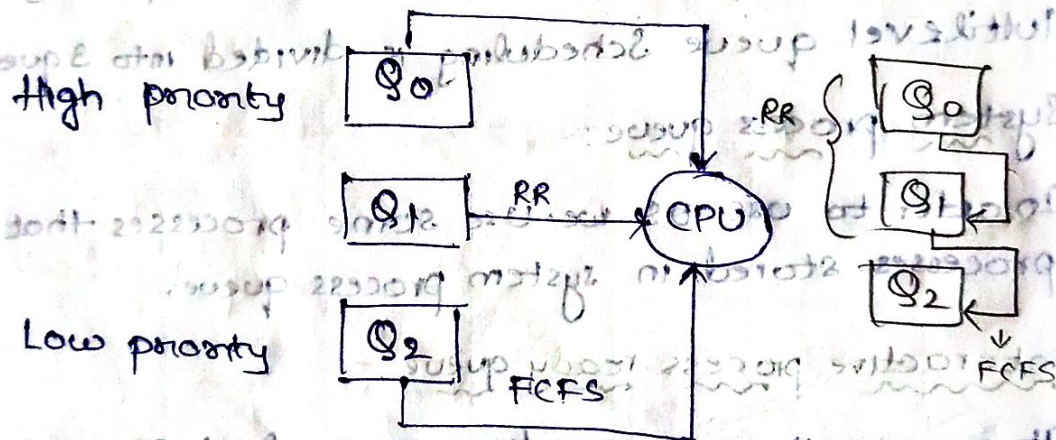
Disadvantages

- With this scheduling the starvation will occur.

Starvation

- Whenever CPU executing process for longer time the lower priority process should wait for longer time. This situation we can call as starvation.
- To overcome the starvation problem there is one more scheduling algorithm is available that is multilevel-feedback scheduling.

Multilevel feedback scheduling



- The processes which are in Q_0, Q_1 will execute with the help of Round Robin.
- The process which are in Q_2 will execute with the help of FCFS.
- First CPU execute all processes in Q_0 then it will execute the processes in Q_1 .
- Here, whenever any new process comes it is placed in Q_0 .

→ According to Round Robin algorithm CPU execute within time quantum. If time quantum exceeds then it is shifted to Q_1 -level.

→ If the process burst time is very high then current process will move to lower level.

FCFS:-

Advantages:-

- 1) It is simple and easy to understand
- 2) FCFS provides fairness by treating all processes equally and giving them an equal opportunity to run.
- 3) FCFS guarantees that every process will eventually get a chance to execute, as long as the system has enough resource to handle all the processes.
- 4) FCFS has low scheduling overhead since it does not involve frequent context switches or complex scheduling decisions.
- 5) FCFS is well-suited for long-running processes or workloads that do not have strict time constraints.

Disadvantages:-

- 1) The process with less execution time suffers i.e. waiting time is often quite long.
- 2) Favors CPU Bound process then I/O bound process.
- 3) This effect results in lower CPU & device utilization.
- 4) FCFS algorithm is particularly troublesome for multiprogramming systems, where it is important that each user get a share of CPU at regular intervals.

2) SJF (Preemptive and Non-preemptive):-

Advantages:-

- 1) Shortest jobs are favored
- 2) It is probably optimal, in that it gives the minimum average WT for a given set of processes.

Disadvantages:-

- 1) SJF may cause starvation if shorter processes keep coming. This problem is solved by aging.
- 2) It cannot be implemented at the level of short term CPU scheduling.

3) Round Robin:-

Advantages:-

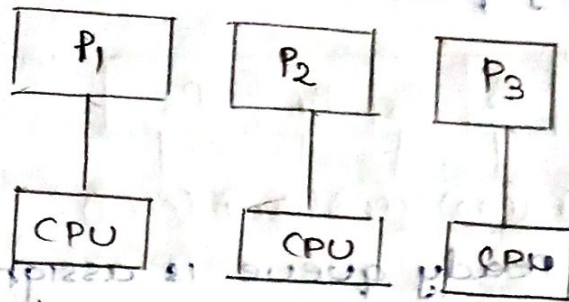
- 1) Every process gets an equal share of CPU.
- 2) RR is cyclic in nature, so there is no starvation.

Disadvantages:-

- 1) Setting the quantum too short increases the overhead and lowers the CPU efficiency, but setting it too long may cause a poor response to short processes.
- 2) The average WT under the RR policy is often long.
- 3) If time quantum is very high then RR degrades to FCFS.

Multiple processor Scheduling :-

→ It means it has more than one processor they work simultaneously with this scheduling the ^{throughput} of the system can be high. But it is more complex than single processor scheduling.



→ There are two approaches to achieve multi processor scheduling

- 1) Asymmetric multiprocessor scheduling (ASMP)
- 2) Symmetric multiprocessor scheduling (SMP)

1) ASMP :-

→ In this ~~sym~~ approach the system activities are handled by single processor i.e. master processor.

→ The remaining all processor work on user instructions.

→ Whenever the master processor fails, then the entire system will fail.

→ The master processor is a central part of system. It shares the memory, hard disk & other memory devices.

2) SMP :-

→ In this instead of single CPU, each processor runs on an individual processor (i.e. CPU)

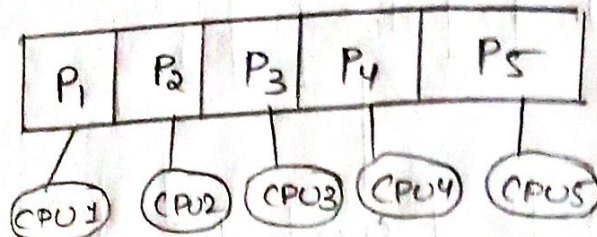
→ There are two advantages with symmetric multiprocessor scheduling.

Global ready queue

Global ready queue

Global ready queue:-

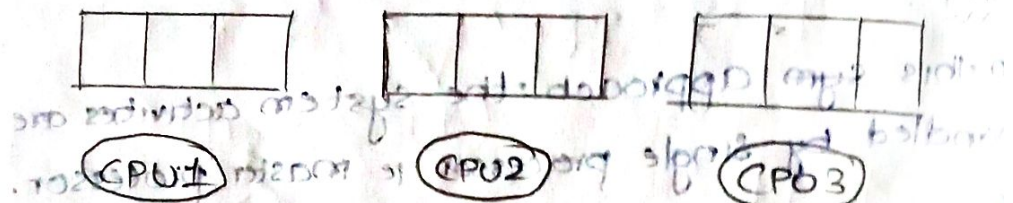
A single ready queue.



In this single ready queue is assigned to the processor.

2) Local ready queue:-

In this each processor will have their own ready queues.



It is involved in high throughput because several tasks are running concurrently in separate processor.

Process affinity

→ If a process runs on a specific processor, there is a certain effects on cache memory.

→ If the process migrate from one processor to another processor, the cache memory of first processor is invalidated and cache memory of second processor is populated.

To avoid this situations, we should keep the process running on the same processor only.

→ This phenomenon is called process affinity.

Load balancing:-

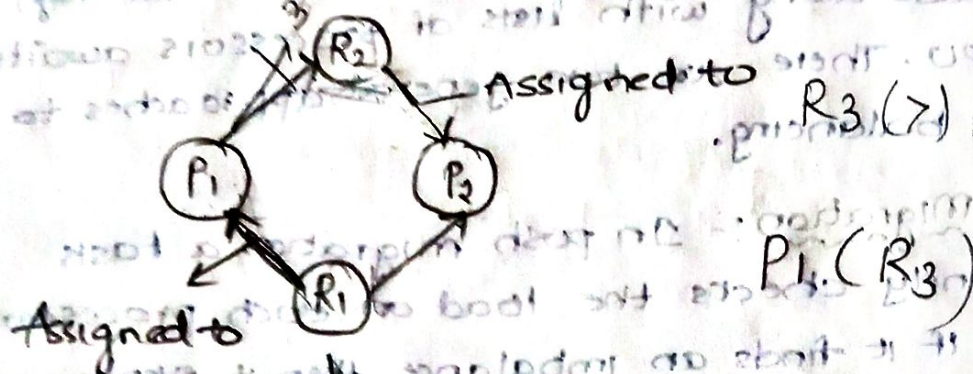
Load balancing is the phenomena which keeps the workload evenly distributed across all processors in an SMP system. Load balancing is necessary only on systems where each processor has its own private queue of process which are eligible to execute. Load balancing is unnecessary because once a processor becomes idle it immediately extracts a runnable process from the common run queue. On SMP, it is important to keep the workload balanced among all processors to fully utilize the benefits of having more than one processor, else one or more processor will sit idle while other processors have high workloads along with lists of processors awaiting the CPU. There are two general approaches to load balancing.

Push Migration:- In push migration a task routinely checks the load on each processor and if it finds an imbalance then it evenly distributes load on each processors by moving the processes from overloaded to idle or less busy processors.

Pull Migration:- Pull migration occurs when an idle processor pulls a waiting task from a busy processor for its execution.

Deadlock

- In OS, Deadlock is a situation where set of processes is blocked because of each process is holding a resource, and wait for another resource which are acquired by another process.
- A process in OS, use the resources in the following way.
 - 1) Request a resource
 - 2) Use the Resource
 - 3) Release the Resource
- A situation occurs in a OS, whenever 2 or more processes hold some resources and wait for the another resource.



There are 4 necessary conditions for deadlock.

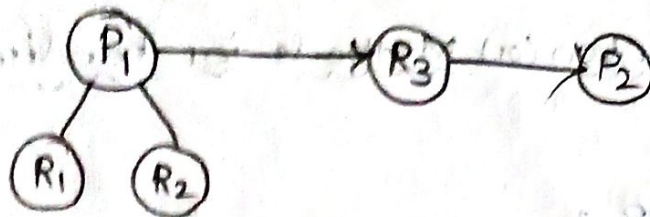
1) Mutual Execution:-

It means a process can use one resource at a time i.e. remaining all resources in non sharable method / state.

Example, Assume that we are sharing the printer for two processes P_1, P_2 then it will give the wrong result so, we have to share one resource at one time.

Hold and wait:-

Consider two process P_1, P_2 . Where P_1 will hold $R_1 \& R_2$ and wait for R_3 which is held by P_2 process.

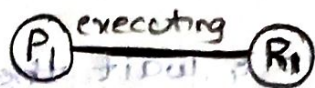


A process is holding some resources and waiting for another resources but those resources are acquired by other processes because of hold & wait the deadlock may occur in the system.

If we eliminate this condition, then there is no deadlock.

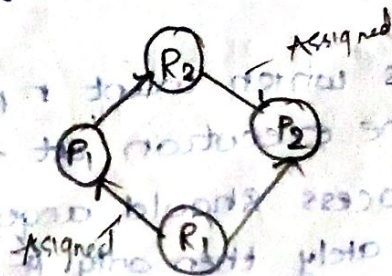
Non Preemption:-

In this CPU, cannot stop the execution of process. A process cannot release the resource until its execution. If we eliminate this condition then we can say that there is no deadlock.



Circular wait:-

A set of processes leads to be deadlock condition because they are waiting for resources which are occupied some by some other processes the entire process will happen in circular way



Note :-

If these are 4 conditions, occurred simultaneously in the system then deadlock may occur.

Deadlock prevention (or) Methods for handling the
Deadlock.

- Deadlock Prevention } Best methods
- Deadlock Avoidance }
- Deadlock Detection
- Deadlock Recovery

Deadlock Prevention :- The Best example is "Prevention is better than the cure" like wise here also, during OS design only the corresponding operations should take necessary actions to prevent the deadlock.

i) Mutual exclusion - In this, a process can use only one resource at a time that's why it is not possible to eliminate mutual exclusion because by default all the resources are in non sharable state.

ii) Hold and wait - In this hold & wait there are 2 approaches in order to eliminate hold and wait.

Approach 1:- A process can request for another resource only if the process is not holding any resources.

Approach 2:- First a process put a request to all the resources.

Consider p is a process which want r resource in order to complete the execution at that

Situation 1st the process should acquire/ occupy all r resources immediately then only it can start the

process.

Note:- With this approach there is a disadvantage that is starvation.

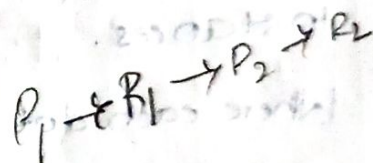
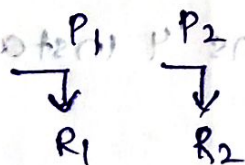
ii) No preemption:- A process can release its resource until unless it has completed

We can eliminate this condition by following 2 approaches

A process put a request for ' R_1 ' resource where the ' R_1 ' is allocated to ' P_3 ' process. Now where ' R_1 ' is not sharable then the OS will check whether the process ' P_1 ' has any other resources (or) not

a) Approach:- let ' P_1 ' request to ' R_1 ' and ' P_2 ' request ' R_2 ' resource. In above situation ' P_1 ' will wait for ' R_1 ' at that time OS forcefully remove the resource ' R_1 ' and allocated that resources to ' P_1 ' then ' P_2 ' is allocated by another resource.

$P_1 - R_1 - P_2 - R_2$



iv) Circular wait:- A circular wait is nothing but each process is waiting in circular order. In order to eliminate circular wait the OS will use mapping function that is $F: R \rightarrow N$ where R is the resource and n is the integer. Here resource is nothing but any printer or scanner or CPU.

For example:- A process holding ' R_1 ' and request for ' R_2 ', if and only if it should satisfy $F(R_1) < F(R_2)$. Now ' P_1 ' equal to ' R_3, R_6, R_4 ' i.e.

$F(R_4) < F(R_3)$

Imp $F(R_C) > F(R_H)$

Deadlock Avoidance

pf Here, we have 2 approaches for Deadlock Avoidance

1) Resource Allocation Graph

2) Banker's Algorithm

1) Resource Allocation Graph

→ Generally, a graph is represented with vertices and edges $G(V, E)$. In this resource allocation graph, the resource type is contain instances. Here, vertices is nothing but process or resource type.

→ A process is represented by circle.

→ Resource type is represented with the help of instances. $\boxed{::}$

→ Where each dot represented as instance

Means eg a process contains 4 instance.

Edges :-

There are mainly 3 types of edges.

1) claim edge

2) request edge

3) Assignment edge.

4) claim edge :-

Whenever a process want the resource in future then it will use claim edge. It is denoted as

$(---\rightarrow)$

2) Request edge:-

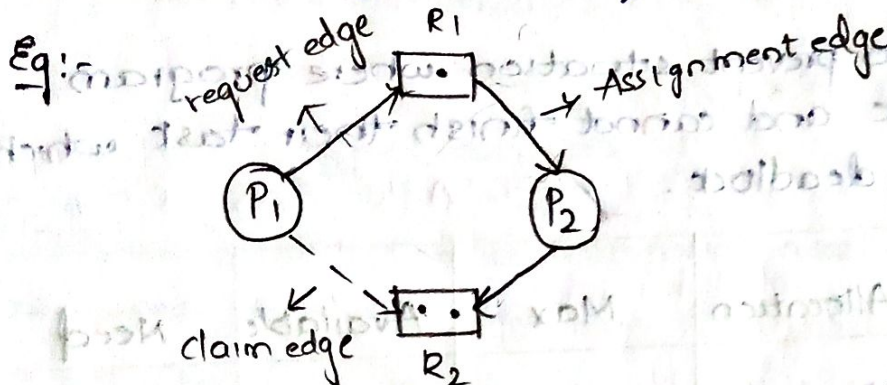
Whenever a process want to use the resource then it put a request to a particular resource.

It is denoted as $(\leftarrow)$

3) Assignment edge:-

Whenever the resource is allocated to particular process then we will use assignment edge.

It is denoted by $(P_2 \leftarrow R_2)$



If a graph contains a cycle then there is possibility of a deadlock.

If there is no cycle in the graph then there is no deadlock.

Note:-

If any resource type contain single instance in the graph then there is a chance of deadlock.

Banker's Algorithm

- It is a deadlock avoidance algorithm. It tests for safety by simulating the allocation for the defined resources.
- It is a smart way for computer system to manage how program will use all resources.
Like CPU, printer, scanner.
- It helps to prevent situation where program get stuck and cannot finish their task which is called deadlock.

$$A=10 \quad B=5 \quad C=7$$

Process	Allocation	Max	Available	Need
P ₀	0 1 0	7 5 3	3 3 2	7 4 3
P ₁	2 0 0	3 2 2		1 2 2
P ₂	3 0 2	9 0 2		6 0 0
P ₃	2 1 1	4 2 2		2 1 1
P ₄	0 0 2	5 3 3		5 3 1

Algorithm

$$work = 3 \ 3 \ 2$$

- 1) $Work = available$ (we have to place those resources in work)
- 2) $finish[i] = false$.
Find an i such that
a) $finish[i] = false$

b) $need \leq work$ (if condition is true, go to step 3)
 3) $work = work + allocation$
 $finish[i] = true$ (executed)
 go to step 2

4) If $finish[i] = true$, for all i , then system in safe state (no deadlock)

Here total a-type resources are 10,

b-type resources are 5,

c-type resources are 7.

Now, we have to find maximum need of resources to total allocation resources

$$a = 7, b = 2, c = 5$$

$$\text{bto Available resources} = 10 - 7 = 3 \rightarrow A$$

$$= 5 - 2 = 3 \rightarrow B$$

$$= 7 - 5 = 2 \rightarrow C$$

Now to get remaining need we have to do max need - allocation.

step-1: $work = Available$ ie $\{3, 3, 2\}$

finish

F	F	F	F	F
0	1	2	3	4

 {The process are not executed}

P₀ :- $Need \leq work$

$$7 \ 4 \ 3 \leq 3 \ 2 \ 2 \text{ ie } \# \text{ is false}$$

So $\#$ must wait.

P₁ :- Need $\leq$ work

122 $\leq$ 332 i.e. it is true.

F	T	F	F	F
---	---	---	---	---

0 1 2 3 4

New work = work + allocation

$$= 332 + 200$$

$$= 532 \text{ [it will execute]}$$

P₂ :- Need $\leq$ work

$$600 \leq 532 \text{ [i.e. false]}$$

P₂ must wait

P₃ :- Need $\leq$ work

$$221 \leq 532 \text{ [i.e. it is false]}$$

Work = work + allocation

$$= 532 + 211$$

$$= 743 \text{ [it will execute]}$$

P₄ :- Need $\leq$ work

$$531 \leq 743 \text{ [i.e. it is true]}$$

New = work + allocation

$$= 743 + 002$$

$$= 745$$

P₀ :- Need $\leq$ work

$$743 \leq 745 \text{ [true]}$$

it will execute

$$\text{work} = \text{work} + \text{allocation}$$

$$= 745 + 010$$

$$\text{E P F} = 755$$

$$P_2: \text{Need} \leq \text{work}$$

$$600 \leq 755 \text{ [true]}$$

$$\text{work} = \text{work} + \text{allocation}$$

$$= 755 + 302$$

$$= 1057$$

Resource Request algorithm:

Step-1:- If request $\leq$ need goto step 2

Step-2:- If request $\leq$ available goto step 3

Step-3:- Allocation = Allocation + request

Available = Available - request

Need = Need - request

Now A=10, B=5, C=7,

P₁ Request for (1,0,2)

$$102 \leq 122 \text{ (true)}$$

$$102 \leq 332 \text{ (true)}$$

$$\text{Allocation} = 200 + 102 = 302$$

$$\text{Available} = 332 - 102 = 230$$

$$\text{Need} = 122 - 102 = 020$$

E P F =

Process	Allocation	Max	Available	need
P ₀	0 1 0	7 5 3	2 3 0	7 4 3
P ₁	3 0 2	3 2 2		0 2 0
P ₂	3 0 2	9 0 2		6 0 0
P ₃	2 1 1	4 2 2		2 1 1
P ₄	0 0 2	5 3 3		5 3 1

P₀ :- Need $\leq$ work

$$7\ 4\ 3 \leq 2\ 3\ 0 \text{ (false)}$$

P₀ should wait

P₁ :- Need $\leq$ work

$$0\ 2\ 0 \leq 2\ 3\ 0 \text{ (true)}$$

$$\begin{aligned} \text{Work} &= \text{work} + \text{allocation} \\ &= 2\ 3\ 0 + 3\ 0\ 2 \\ &= 5\ 3\ 2 \end{aligned}$$

P₂ :- Need $\leq$ work

$$6\ 0\ 0 \leq 5\ 3\ 2 \text{ (false)}$$

P₂ should wait

P₃ :- Need $\leq$ work

$$2\ 1\ 1 \leq 5\ 3\ 2 \text{ (true)}$$

$$\begin{aligned} \text{Work} &= \text{work} + \text{allocation} \\ &= 5\ 3\ 2 + 2\ 1\ 1 \\ &= 7\ 4\ 3 \end{aligned}$$

P₁ :- Need $\leq$ work

$$531 \leq 743 \text{ (true)}$$

$$\text{work} = \text{work} + \text{allocation}$$

$$= 743 + 002$$

$$= 745$$

P₀ :- Need $\leq$ work

$$743 \leq 745 \text{ (true)}$$

$$\text{work} = \text{work} + \text{allocation}$$

$$= 745 + 0101$$

$$= 755$$

P₂ :- Need $\leq$ work

$$000 \leq 755 \text{ (true)}$$

$$\text{work} = \text{work} + \text{allocation}$$

$$= 755 + 302$$

$$= 1057$$

Deadlock Detection

Wait for graph [single instance] :-

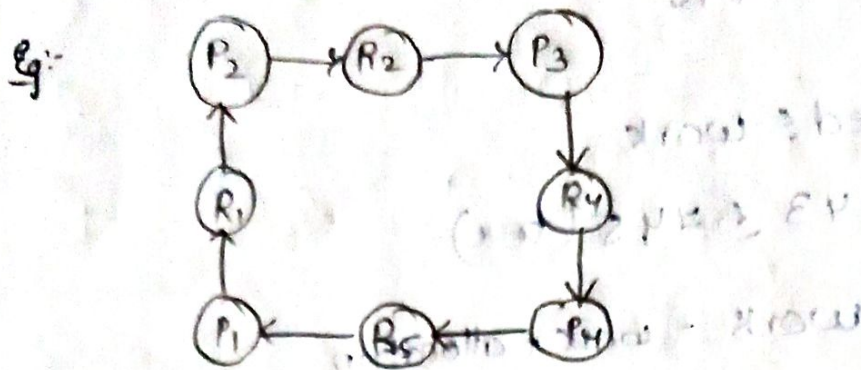
It is a mechanism of detecting and resolving deadlock in OS. In OS deadlock detection and

Recovery is very important to keep everything running smoothly.

There are two approaches for deadlock detection.

- 1) Single instance (construction of wait for graph)

- If the graph contain single instance then we have to use resource allocation graph. In this method all the processes are allocated to resources.
- To construct wait for graph first we have to remove the resources b/w the processes.



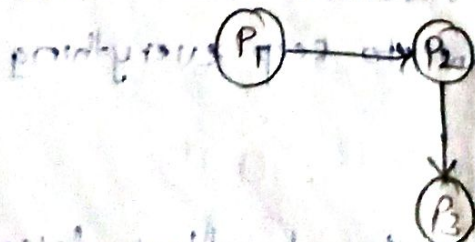
In above graph P_1 is requesting for R_1 but R_1 is assigned to P_2 . Similarly P_2 is requesting for R_2 and R_2 is assigned to P_3 . Like this all the resources are allocated to respective processes.

Wait for graph:-

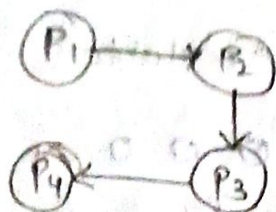
Step-1:- First remove the R_1 resource which is allocated to P_2 . then construct an edge from P_1 to P_2 .



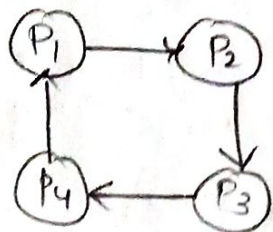
Step-2:- Remove the resource R_2 then construct an edge between P_2 to P_3 . Now.



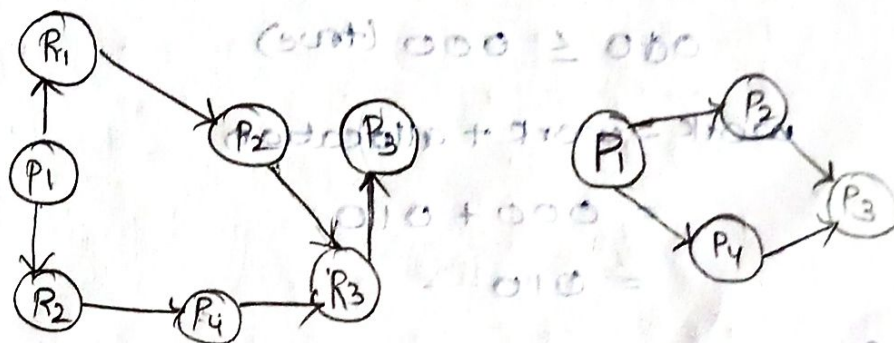
Step-3:- Now remove the resource R_4 then construct an edge between P_3 and P_4



Step-4:- Now remove the resource R_5 then construct an edge between P_4 and P_1 .



By constructing wait for graph if we get closed loop then there is a deadlock.



Multiple instance of resource type

Algorithm:-

- 1) Work = available
 finish[i] = false.
 find i such that
 - 2) finish[i] = false
 Request $\leq$ work.
- 3) Work = Work + allocation
 finish[i] = true; then go to step-2.
- 4) If finish[i] = false for all process then system is in unsafe state.

	Allocation	Request	Available
P ₀	0 1 0	0 0 0	0 0 0
P ₁	2 0 0	2 0 2	
P ₂	3 0 3	0 0 0	
P ₃	2 1 1	1 0 0	
P ₄	0 0 2	0 0 2	

work

P₀ :- work = 0 0 0
Request ≤ work

$$0 0 0 \leq 0 0 0 \text{ (true)}$$

$$\begin{aligned} \text{work} &= \text{work} + \text{allocation} \\ &= 0 0 0 + 0 1 0 \\ &= 0 1 0 \end{aligned}$$

P₁ :- work = 0 1 0

$$\text{Request} \leq \text{work}$$

$$2 0 2 \leq 0 1 0 \text{ (false)}$$

P₂ :- Request ≤ work

$$0 0 0 \leq 0 1 0 \text{ (true)}$$

$$\begin{aligned} \text{work} &= \text{work} + \text{allocation} \\ &= 0 1 0 + 3 0 3 \\ &= 3 1 3 \end{aligned}$$

P₃ :-

$$\text{work} = 3 1 3$$

$$\text{Request} \leq \text{work (true)}$$

$$1 0 0 \leq 3 1 3$$

$$\text{work} = \text{work} + \text{allocation}$$

$$= 313 + 211$$

$$= 524$$

$$P_4 :- \text{work} = 524$$

$$\text{Request} \leq \text{work}$$

$$002 \leq 524 (\text{true})$$

$$\text{work} = \text{work} + \text{allocation}$$

$$= 524 + 002$$

$$= 526$$

$$P_1 :- \text{work} = 526$$

$$\text{Request} \leq \text{work}$$

$$202 \leq 526 (\text{true})$$

$$\text{work} = \text{work} + \text{allocation}$$

$$= 526 + 200$$

$$= 726$$

Imp

Deadlock Recovery:-

Whenever deadlock is detected, in the system then we have to resolve the system by following approaches.

1) Process termination:-

About all the deadlocked processes it means out of 10 process 3 are deadlocked for those processes we have to use kill mechanism. But the 3 processes are already executed some amount of time, with this we are wasting the resources of those processes

that's why that is expensive approach.

2nd approach:- kill the process at a time until deadlock cycle is eliminated. First we have to kill the first process then apply banker's algorithm for that to check whether all resources are allocated properly or not. All By doing all these steps too much amount of time is going to waste that's why kill the process which is executed in less amount of time.

3) Resource preemptive:-

Preemption means releasing the resource. So, we have to release the resource from corresponding process and allocate that resource to another process.

There are 3 issues to be considered while releasing the resource for the process.

Selecting a victim:- we must determine which resources and which processes are to be preempted and also in order to minimize the cost.

Roll back:- we must determine what should be done with the processes from which resources are preempted. One simple idea is total rollback. That means aborting the process and restarting it.

Starvation:- In a system, it may happen that same process is always picked as a victim. As a result, that process will never complete its designated task. This situation is called starvation and must be avoided. One solution is that a process must be picked as victim only a finite number of times.